



Docker Basics: Images and Containers

A practical guide for AI engineers and ML researchers adopting containerized workflows

Why Docker for AI?

Docker has become an essential tool in the AI engineering toolkit because it:

Eliminates environment inconsistencies

No more "it works on my machine" problems when sharing models or deploying to production

Packages complete environments

Code, dependencies, and configurations travel together as one portable unit

Enables reproducible experiments

Critical for scientific validity and debugging in complex ML systems



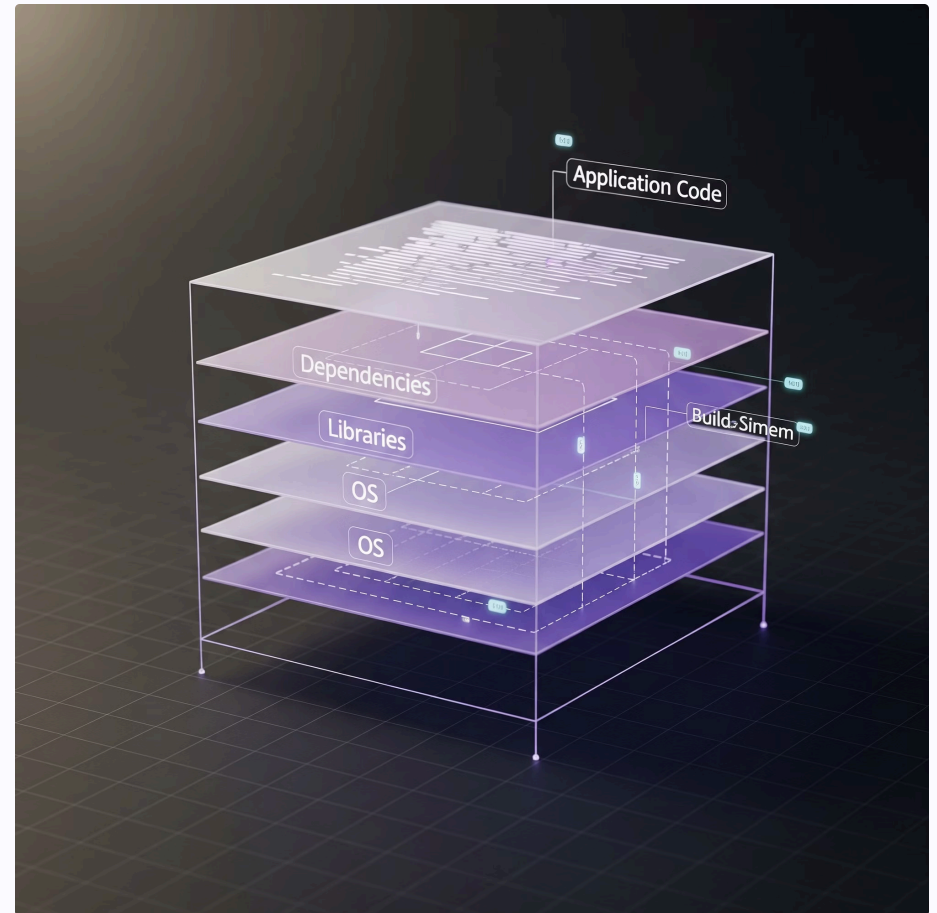
What Is a Docker Image?

A blueprint for running applications

Docker images are the foundation of containerization, providing:

- A complete package containing OS libraries, dependencies, configurations, and code
- Immutability once built, ensuring consistency across environments
- Shareable artifacts via registries like Docker Hub, Amazon ECR, or Google GCR

Example: `pytorch/pytorch:2.1.0-cuda12.1-cudnn8`



What Is a Docker Container?

Running Instance

A container is a running instance of a Docker image - lightweight and isolated from other processes on the host OS

Lifecycle Management

Containers can be started, stopped, paused, or deleted as needed without affecting the host system

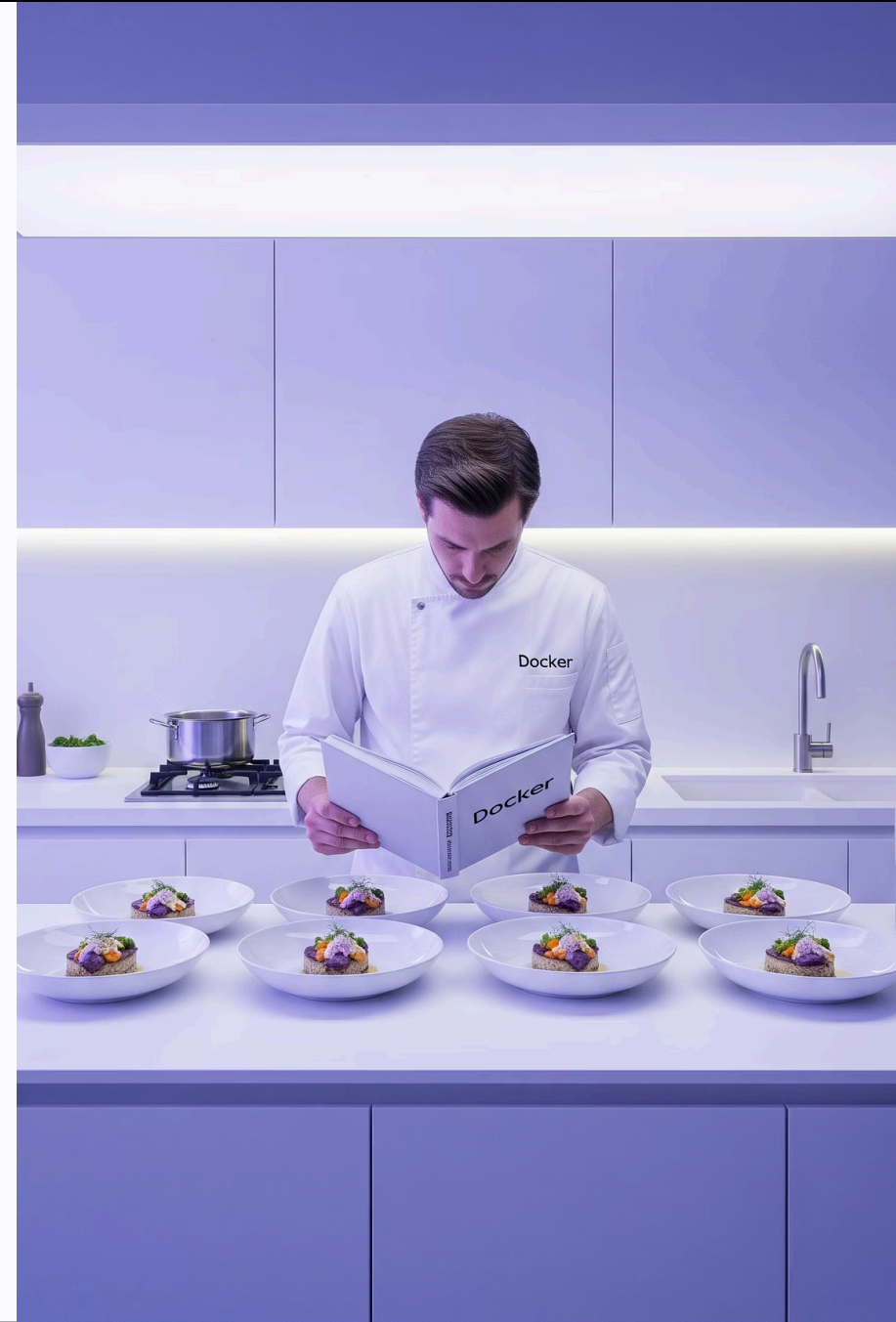
Resource Efficiency

Multiple containers can run simultaneously on one machine, each with its own isolated environment

The Perfect Analogy

Image = Recipe (instructions for creating something)

Container = Prepared Meal (the actual instance you can use)



Docker Workflow Basics



Build

Create an image from a Dockerfile using `docker build`



Run

Start a container from an image using `docker run`



Share

Distribute images via registries using `docker push/pull`



Deploy

Orchestrate containers in production using Kubernetes

This simple but powerful workflow scales from individual development to enterprise MLOps, making Docker the backbone of modern AI deployment pipelines.



Key Docker Commands

List all images on your system

```
docker images
```

List all containers (including stopped)

```
docker ps -a
```

Build an image from a Dockerfile

```
docker build -t myimage .
```

Run a container interactively

```
docker run -it myimage
```

Enter a running container

```
docker exec -it container_id bash
```

Command Structure

Docker commands follow a consistent pattern:

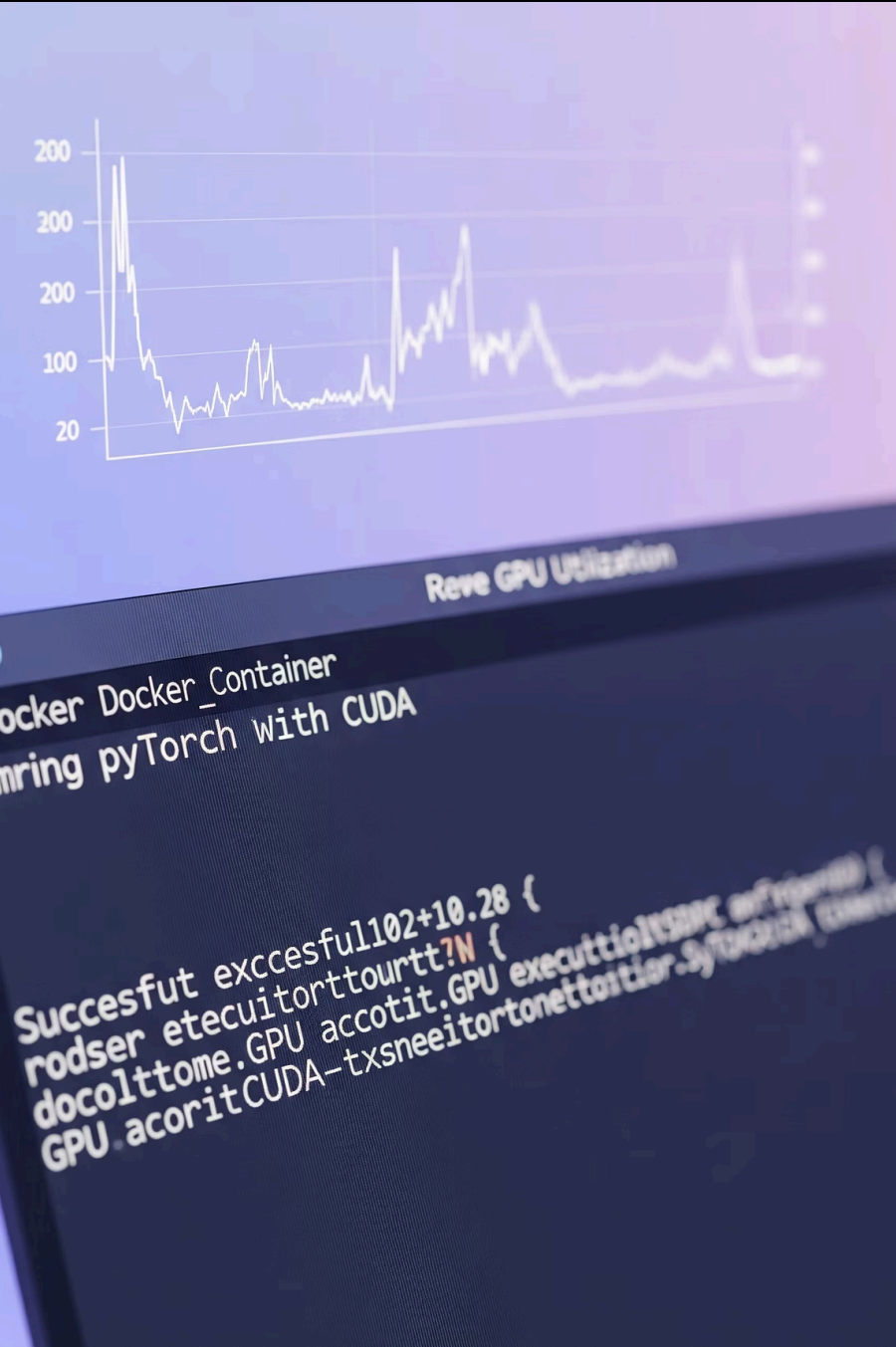
```
docker [command] [options] [arguments]
```

Getting Help

Access documentation for any command:

```
docker [command] --help
```

 These five commands will handle 80% of your daily Docker interactions when working with AI models and applications.



Example – Running a PyTorch Container

```
docker run --gpus all -it \\\npytorch/pytorch:2.1.0-cuda12.1-cudnn8 \\\nbash
```

What's Happening

- Pulls the official PyTorch image from Docker Hub
- Maps host GPUs to the container
- Launches interactive bash shell

Requirements

- Docker Engine installed
- NVIDIA Container Toolkit
- NVIDIA drivers on host

Benefits

- Isolated PyTorch environment
- Full GPU acceleration
- Zero configuration needed



Dockerfile – Build Your Own Image

A Dockerfile is a text file containing instructions to build a custom Docker image:

```
FROM python:3.10-slim
```

```
RUN pip install torch torchvision
```

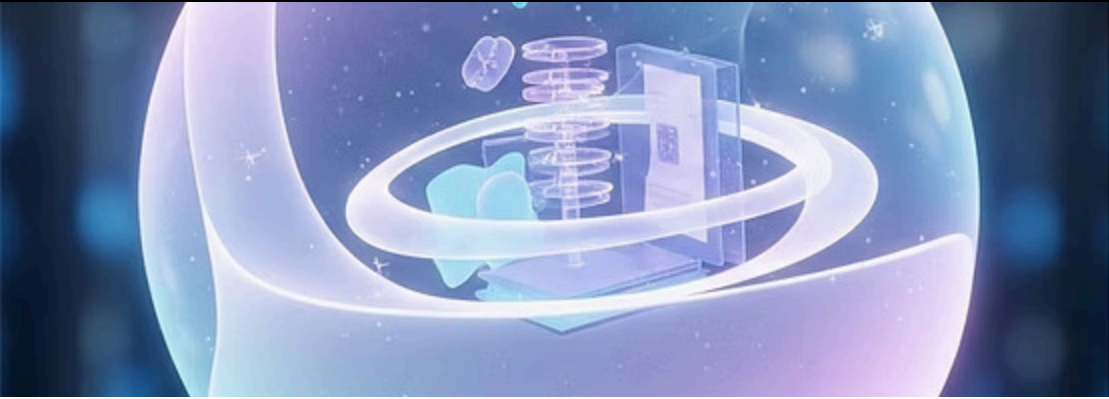
```
COPY app.py /app/app.py
```

```
CMD ["python", "/app/app.py"]
```

Build with: `docker build -t myai:latest .`

Common Instructions

- **FROM:** Base image to build upon
- **RUN:** Execute commands in the image
- **COPY:** Add files from host to image
- **WORKDIR:** Set working directory
- **ENV:** Set environment variables
- **EXPOSE:** Document network ports
- **CMD:** Default command to run



Containers vs. Virtual Environments

Virtual Environments

- Python-only isolation (virtualenv, conda)
- Lightweight and simple to set up
- Same OS and system libraries
- Limited portability across systems
- No isolation for non-Python dependencies

Docker Containers

- Complete system-level isolation
- Packages OS libraries, CUDA drivers, etc.
- Highly portable across systems
- Shareable with exact reproducibility
- Works consistently in production

For AI workloads, containers provide stronger guarantees for reproducibility and deployment consistency, especially for complex ML stacks.

Best Practices for AI Containers

Use Official Base Images

Start with trusted images like `pytorch/pytorch` or `tensorflow/tensorflow` rather than building from scratch

Keep Images Small

Use multi-stage builds to eliminate build tools, test data, and unnecessary libraries from final images

Pin Specific Versions

Always specify exact versions of base images and dependencies like `pytorch:2.1.0-cuda12.1-cudnn8` not `pytorch:latest`

Don't Run as Root

Create a non-root user inside containers for better security, especially in shared environments

Automate Builds

Implement CI/CD pipelines to automatically build, test, and push images when code changes

- ❑ These practices will save you hours of debugging and make your ML workflows more robust across both research and production environments.

Key Takeaways



Remember:

- **Images are blueprints**, containers are running instances
- Docker ensures **portability & reproducibility** for AI workloads
- Docker is **essential infrastructure** for modern MLOps
- GPU acceleration works **seamlessly** via NVIDIA Container Toolkit
- Containerization is the **foundation** for scaling with Kubernetes

Next Steps:

Practice building custom images for your ML workflows and explore Docker Compose for multi-container applications.